

Examen Unidad 5: Tratamiento de archivos con Java

Realizar cada ejercicio en un proyecto independiente.

Entregar un archivo comprimido `.zip` por cada ejercicio.

El archivo comprimido corresponderá a la carpeta completa del proyecto.

Se valorará positivamente:

- La correcta organización del código en clases.
- El uso adecuado de Jackson para leer y escribir archivos JSON.
- El uso de `try-with-resources`.
- La claridad de los nombres de variables y métodos.
- La correcta gestión de excepciones.
- Que el programa no finalice bruscamente ante errores previsibles.

Ejercicio 1 - Gestión de Cursos de Formación

Puntuación: 5 puntos

Desarrollar un programa en Java que lea un archivo JSON con información de varios cursos de formación, actualice algunos datos según determinados criterios, filtre los cursos que cumplan unas condiciones concretas y genere tanto un nuevo archivo JSON como un informe en formato texto.

Se proporciona un archivo llamado `cursos.json` que contiene información sobre varios cursos. Cada curso tiene un nombre, profesor, número de horas, precio y modalidad.

Clase Curso

Se deberá crear una clase `Curso` con los siguientes atributos:

- `nombre` (`String`): nombre del curso.
- `profesor` (`String`): profesor responsable del curso.
- `horas` (`int`): duración del curso en horas.
- `precio` (`double`): precio del curso.
- `modalidad` (`String`): modalidad del curso, por ejemplo `"Online"` o `"Presencial"`.

La clase deberá incluir aquellos métodos necesarios.

Archivo inicial `cursos.json`

```

1  [
2    {
3      "nombre": "Introducción a Java",
4      "profesor": "Laura",
5      "horas": 50,
6      "precio": 250,
7      "modalidad": "Online"
8    },
9    {
10     "nombre": "Bases de Datos con MySQL",
11     "profesor": "Carlos",
12     "horas": 60,
13     "precio": 320,
14     "modalidad": "Presencial"
15   },
16   {
17     "nombre": "Desarrollo Web con HTML y CSS",
18     "profesor": "Laura",
19     "horas": 35,
20     "precio": 180,
21     "modalidad": "Online"
22   },
23   {
24     "nombre": "Programación Avanzada en Java",
25     "profesor": "Miguel",
26     "horas": 80,
27     "precio": 450,
28     "modalidad": "Presencial"
29   },
30   {
31     "nombre": "Administración de Sistemas Linux",
32     "profesor": "Ana",
33     "horas": 70,
34     "precio": 390,
35     "modalidad": "Online"
36   },
37   {
38     "nombre": "Programación Básica en Java",
39     "profesor": "Miguel",
40     "horas": 50,
41     "precio": 150,
42     "modalidad": "Online"
43   },
44   {
45     "nombre": "Ciberseguridad Básica",
46     "profesor": "Carlos",
47     "horas": 50,
48     "precio": 300,
49     "modalidad": "Online"
50   },
51 ]

```

```

52     {
53         "nombre": "Aplicaciones Web con Java",
54         "profesor": "Laura",
55         "horas": 90,
56         "precio": 500,
57         "modalidad": "Presencial"
58     },
59     {
60         "nombre": "Introducción a Python",
61         "profesor": "Sergio",
62         "horas": 45,
63         "precio": 220,
64         "modalidad": "Online"
65     },
66     {
67         "nombre": "Java SpringBoot",
68         "profesor": "Miguel",
69         "horas": 100,
70         "precio": 550,
71         "modalidad": "Online"
72     }
73 ]

```

Tareas a realizar

1. Leer el archivo `cursos.json`

El programa deberá leer el contenido del archivo `cursos.json` y cargar los datos en una lista de objetos `Curso`.

Puntuación: 0,5 puntos

Se valorará positivamente el uso de `try-with-resources` cuando proceda.

2. Actualizar los cursos impartidos por Laura

Todos los cursos cuyo profesor sea `"Laura"` deberán pasar a tener como profesor a `"Lucía"`.

Puntuación: 0,5 puntos

3. Seleccionar los cursos online de más de 45 horas

El programa deberá crear una nueva lista ordenada por el número de horas (de mayor a menor) que contenga únicamente los cursos que cumplan las dos condiciones siguientes:

- Su modalidad es `"Online"`.
- Tienen más de 45 horas.
- Su título contiene la palabra *Java*.

Puntuación: 1,25 puntos.

4. Sobrescribir el archivo `cursos.json`

Una vez actualizados los profesores, el programa deberá sobrescribir el archivo `cursos.json` con la lista completa de cursos actualizada.

Puntuación: 0,5 puntos.

El archivo original deberá quedar modificado con los cambios realizados.

5. Generar un nuevo archivo JSON con los cursos filtrados

El programa deberá generar un nuevo archivo llamado: `cursos_Java_online_largos.json`. Este archivo deberá contener únicamente los cursos online de más de 45 horas cuyo título contenga la palabra *Java*.

Puntuación: 0,5 puntos

6. Generar un informe de texto

Además del archivo JSON anterior, el programa deberá generar un archivo de texto llamado: `informe_cursos_Java_largos_online.txt`. Este archivo deberá contener un resumen legible de los cursos seleccionados. El **formato** del archivo (NO EL CONTENIDO) será similar al siguiente:

```
1  Nombre: Administración de Sistemas Linux
2  Profesor: Ana
3  Horas: 70
4  Precio: 390.0 €
5  Modalidad: Online
6
7  Nombre: Ciberseguridad Básica
8  Profesor: Carlos
9  Horas: 50
10 Precio: 300.0 €
11 Modalidad: Online
12
13 ...
```

Puntuación: 0,75 puntos

7. Mostrar por pantalla los resultados

El programa deberá mostrar por pantalla:

- La lista completa de cursos actualizada.
- La lista de cursos online de más de 45 horas cuyo título contenga la palabra Java.
- Un mensaje indicando que los archivos se han generado correctamente.

Puntuación: 0,75 puntos

Ejercicio 2 - Gestión de Reservas de Aulas

Puntuación: 5 puntos

Objetivo

Desarrollar una aplicación para gestionar reservas de aulas utilizando un mapa, donde cada reserva estará asociada a un identificador único.

Los datos deberán cargarse desde un archivo JSON llamado `reservas.json`.

El programa deberá:

- Cargar las reservas desde JSON.
- Añadir nuevas reservas solicitando los datos por teclado.
- Listar todas las reservas.
- Eliminar reservas por identificador.
- Guardar los cambios en el archivo JSON.
- Generar un archivo de texto con el listado de reservas.
- Crear y utilizar una excepción personalizada.
- Generar un archivo JSON con todas las reservas agrupadas por profesor.

El programa tiene una parte interactiva que se deberá gestionar mediante el correspondiente menú y su correspondiente opción de *Terminar Programa*.

Clase Reserva

Se deberá crear una clase `Reserva` con los siguientes atributos:

- `id` (`String`): identificador único de la reserva.
- `aula` (`String`): nombre o número del aula.
- `profesor` (`String`): nombre del profesor que realiza la reserva.
- `fecha` (`String`): fecha de la reserva, por ejemplo `"2026-05-10"`.
- `hora` (`String`): hora de la reserva, por ejemplo `"10:00"`.
- `duracion` (`int`): duración de la reserva en minutos.

La clase deberá incluir aquellos métodos necesarios.

Archivo inicial `reservas.json`

```

1  {
2    "R001": {
3      "id": "R001",
4      "aula": "Aula 101",
5      "profesor": "María",
6      "fecha": "2026-05-10",
7      "hora": "09:00",
8      "duracion": 60
9    },
10   "R002": {
11     "id": "R002",
12     "aula": "Aula 203",
13     "profesor": "Juan",
14     "fecha": "2026-05-10",
15     "hora": "11:30",
16     "duracion": 90
17   },
18   "R003": {
19     "id": "R003",
20     "aula": "Laboratorio 1",
21     "profesor": "Ana",
22     "fecha": "2026-05-11",
23     "hora": "08:30",
24     "duracion": 120
25   },
26   "R004": {
27     "id": "R004",
28     "aula": "Aula 105",
29     "profesor": "Carlos",
30     "fecha": "2026-05-11",
31     "hora": "10:00",
32     "duracion": 60
33   },
34   "R005": {
35     "id": "R005",
36     "aula": "Laboratorio 2",
37     "profesor": "María",
38     "fecha": "2026-05-12",
39     "hora": "12:00",
40     "duracion": 90
41   },
42   "R006": {
43     "id": "R006",
44     "aula": "Aula 301",
45     "profesor": "Juan",
46     "fecha": "2026-05-12",
47     "hora": "08:15",
48     "duracion": 45
49   },
50   "R007": {
51

```

```

52     "id": "R007",
53     "aula": "Salón de actos",
54     "profesor": "Ana",
55     "fecha": "2026-05-13",
56     "hora": "11:00",
57     "duracion": 120
58 },
59 "R008": {
60     "id": "R008",
61     "aula": "Aula 204",
62     "profesor": "Miguel",
63     "fecha": "2026-05-13",
64     "hora": "09:30",
65     "duracion": 60
66 },
67 "R009": {
68     "id": "R009",
69     "aula": "Laboratorio 3",
70     "profesor": "María",
71     "fecha": "2026-05-14",
72     "hora": "12:30",
73     "duracion": 90
74 },
75 "R010": {
76     "id": "R010",
77     "aula": "Aula 102",
78     "profesor": "Carlos",
79     "fecha": "2026-05-14",
80     "hora": "08:30",
81     "duracion": 60
82 },
83 "R011": {
84     "id": "R011",
85     "aula": "Aula 205",
86     "profesor": "Juan",
87     "fecha": "2026-05-15",
88     "hora": "10:30",
89     "duracion": 90
90 },
91 "R012": {
92     "id": "R012",
93     "aula": "Laboratorio 1",
94     "profesor": "Ana",
95     "fecha": "2026-05-15",
96     "hora": "09:00",
97     "duracion": 120
98 },
99 "R013": {
100     "id": "R013",
101     "aula": "Aula 302",
102

```

```

103     "profesor": "Miguel",
104     "fecha": "2026-05-18",
105     "hora": "12:00",
106     "duracion": 60
107 },
108 "R014": {
109     "id": "R014",
110     "aula": "Biblioteca",
111     "profesor": "María",
112     "fecha": "2026-05-18",
113     "hora": "10:00",
114     "duracion": 45
115 },
116 "R015": {
117     "id": "R015",
118     "aula": "Aula 103",
119     "profesor": "Carlos",
120     "fecha": "2026-05-19",
121     "hora": "13:00",
122     "duracion": 60
123 },
124 "R016": {
125     "id": "R016",
126     "aula": "Laboratorio 2",
127     "profesor": "Juan",
128     "fecha": "2026-05-19",
129     "hora": "08:30",
130     "duracion": 120
131 },
132 "R017": {
133     "id": "R017",
134     "aula": "Aula 201",
135     "profesor": "Ana",
136     "fecha": "2026-05-20",
137     "hora": "11:00",
138     "duracion": 90
139 },
140 "R018": {
141     "id": "R018",
142     "aula": "Aula 106",
143     "profesor": "Miguel",
144     "fecha": "2026-05-20",
145     "hora": "09:00",
146     "duracion": 60
147 },
148 "R019": {
149     "id": "R019",
150     "aula": "Laboratorio 3",
151     "profesor": "María",
152     "fecha": "2026-05-21",
153

```

```

154     "hora": "12:00",
155     "duracion": 120
156 },
157 "R020": {
158     "id": "R020",
159     "aula": "Aula 104",
160     "profesor": "Carlos",
161     "fecha": "2026-05-21",
162     "hora": "10:30",
163     "duracion": 90
164 }
165 }

```

Clase GestorReservas

Se deberá crear una clase `GestorReservas` que gestione un mapa `Map<String, Reserva>` reservas. La clave del mapa será el identificador de la reserva.

Excepción personalizada

Se deberá crear una excepción personalizada llamada:

`ReservaDuplicadaException`

Esta excepción deberá lanzarse cuando se intente añadir una reserva cuyo identificador ya exista en el mapa.

Por ejemplo, si ya existe la reserva `"R001"` y el usuario intenta añadir otra reserva con el mismo identificador, el programa deberá lanzar y gestionar la excepción.

Tareas a realizar

1. Leer y cargar las reservas desde `reservas.json`

El programa deberá leer el archivo `reservas.json` y cargar su contenido en un `Map<String, Reserva>`.

Puntuación: 0,25 puntos

2. Crear la excepción personalizada `ReservaDuplicadaException`

Se deberá crear, lanzar y capturar correctamente una excepción personalizada que se utilizará cuando se intente introducir una reserva con un identificador repetido.

Puntuación: 1,5 puntos

3. Añadir nuevas reservas por teclado

El programa deberá permitir añadir nuevas reservas solicitando por teclado:

- Identificador.
- Aula.
- Profesor.
- Fecha.
- Hora.
- Duración.

Puntuación: 0,5 puntos

4. Listar todas las reservas existentes

El programa deberá mostrar por pantalla todas las reservas almacenadas en el mapa.

Puntuación: 0,5 puntos

5. Eliminar una reserva por identificador

El programa deberá permitir eliminar una reserva solicitando al usuario su identificador. Si el identificador existe, la reserva se eliminará del mapa.

Si no existe, se mostrará un mensaje informativo.

Puntuación: 0,75 puntos

6. Guardar el estado actualizado en `reservas.json`

Después de realizar modificaciones, el programa deberá guardar el contenido actualizado del mapa en el archivo `reservas.json`.

Puntuación: 0,25 puntos

7. El programa deberá generar un archivo llamado: `listado_reservas.txt`.

Este archivo deberá contener todas las reservas almacenadas en el mapa en un formato legible.

Puntuación: 0,25 puntos

Ejemplo:

```
1 LISTADO DE RESERVAS DE AULAS
2 ID: R001
3 Aula: Aula 101
4 Profesor: María
5 Fecha: 2026-05-10
6 Hora: 09:00
7 Duración: 60 minutos
8
9 ID: R002
10 Aula: Aula 203
11 Profesor: Juan
12 Fecha: 2026-05-10
13 Hora: 11:30
14 Duración: 90 minutos
15
16 ...
```

8. Generar un archivo JSON llamado `reservas_por_profesor.json` que contenga un mapa donde la clave será el nombre del profesor y el valor una lista de todas las reservas de dicho profesor. Puntuación: **1 punto**.